

使用判斷敘述與迴圈敘述

- **18-1:判斷敘述**

選擇結構.....	18-1
if敘述.....	18-2
三元運算式.....	18-5
串連式if敘述.....	18-6
巢狀if敘述.....	18-7
switch敘述.....	18-8

- **18-2:迴圈敘述**

while迴圈.....	18-11
do迴圈.....	18-13
for迴圈.....	18-15
foreach迴圈.....	18-17
goto敘述.....	18-19
goto敘述使用在switch.....	18-20
break敘述.....	18-21
continue敘述.....	18-22

Module 18

選擇結構

- 程式的三大基本結構：循序、選擇和重複結構，程式由三者所提供的敘述組合而成。
- 循序結構的特性是由上而下逐行執行。
- 選擇結構是當程式執行時，根據條件判斷，決定程式執行的路徑。
- 重複結構又稱迴圈，當程式某段敘述需重複執行多次時使用。
- 選擇結構利用條件判斷式為程式進行流程控制。

if敘述-1

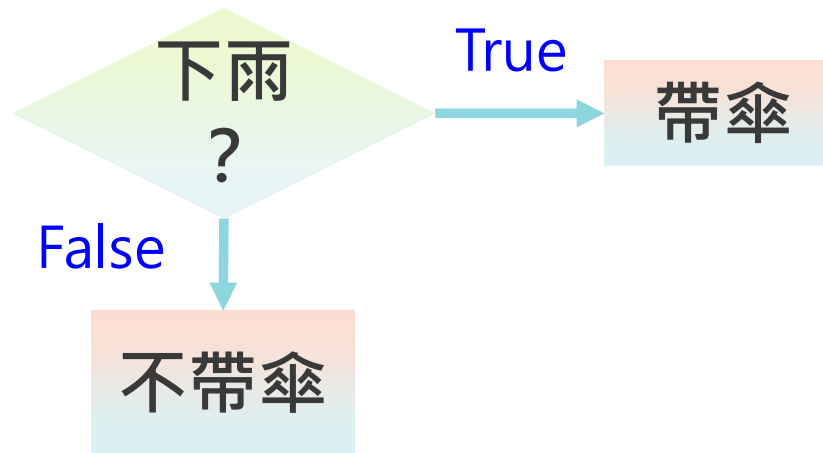
- if敘述是流程控制的基本語法之一。
- 範例：

— **if** (下雨)

帶傘;

else

不帶傘;



if敘述-2

- 單if敘述：

- **if** (運算式){

- //...運算式結果為true的程式碼...

- }

- if...else...敘述：

- **if** (運算式){

- //...運算式結果為true的程式碼...

- }

- else**{

- //...運算式結果為false的程式碼...

- }

if敘述-3

- if敘述 () 內的運算結果必須是布林值。
- 須注意型態轉換。

```
int a = 11;  
if (a)  
{  
    Console.WriteLine(a);  
}
```

❏	程式碼	說明
❌	CS0029	無法將類型 'int' 隱含轉換成 'bool'

三元運算式

- 三元運算式讓if...else敘述更佳簡潔。

```
bool isRainy = true;  
string result;  
if (isRainy)  
{  
    result = "帶傘";  
}  
else  
{  
    result = "不帶傘";  
}
```



```
bool isRainy = true;  
string result;  
result = isRainy ? "帶傘" : "不帶傘";
```

串連式if敘述

- 串連式if敘述：
 - if** (運算式1){
 //...運算式1結果為true的程式碼...
}
 - else if** (運算式2){
 //...運算式1結果為false，但運算式2結果為true的程式碼...
}
 - else**{
 //...運算式2結果為false的程式碼...
}

巢狀if敘述

- 巢狀if敘述：

- if** (運算式1){
 //...運算式結果為true的程式碼...
 if (運算式2){
 //...運算式2結果為true的程式碼...
 }
}
else{
 //...運算式結果為false的程式碼...
}

switch敘述-1

- 適合用來處理多種狀況判斷。
- 比起if敘述，某些情況下switch敘述看起來更簡潔。
- 簡單來說，switch依判斷式或變數的值，在所有預先寫好的情況(case)中，選擇符合條件的敘述執行。
- 每個case的結尾必須加上break;表示結束。

switch敘述-2

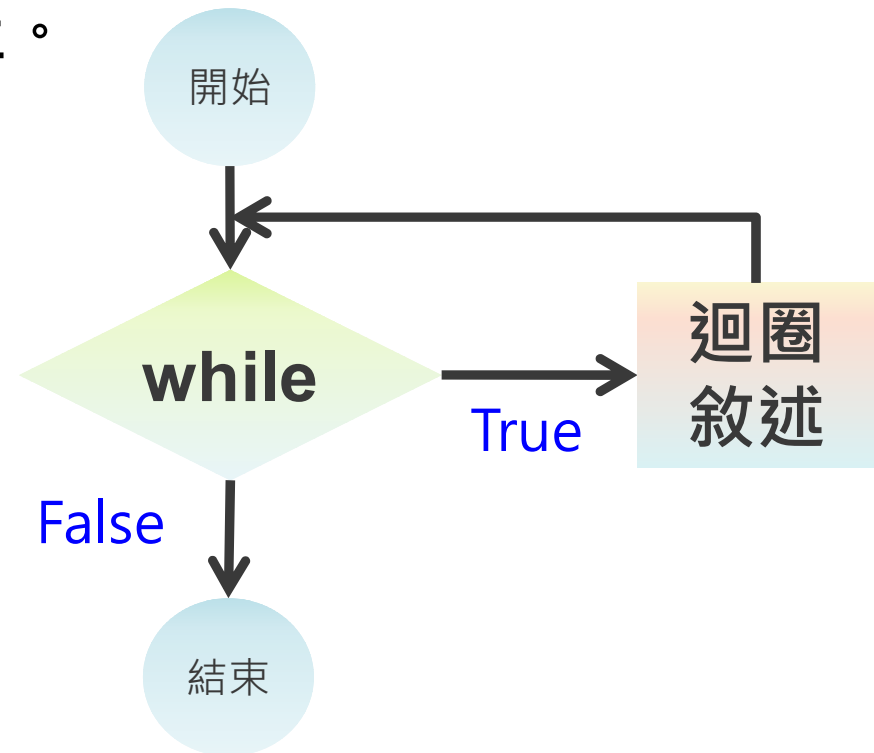
- 語法：
 - **switch** (要判斷的物件){
 - case** 情況1 :
 - //...物件符合情況1會執行的程式碼...
 - break;**
 - case** 情況2 :
 - //...物件符合情況2會執行的程式碼...
 - break;**
 - default** :
 - //...物件不符合以上情況時會執行的程式碼...
 - break;** }

switch敘述-3

```
string isRainy = "下雨";  
string result ;  
switch (isRainy)  
{  
    case "下雨":  
        result = "帶傘";  
        break;  
    case "沒下雨":  
        result = "不帶傘";  
        break;  
    default:  
        result="介於有和沒有之間";  
        break;  
}
```

while 迴圈-1

- **while**為前測式迴圈，**while**後的條件運算式為**true**則進入迴圈執行，直到條件運算式為**false**停止。
- 為前測式迴圈，可能未執行就停止。
- 必須避免產生無窮迴圈。



while 迴圈-2

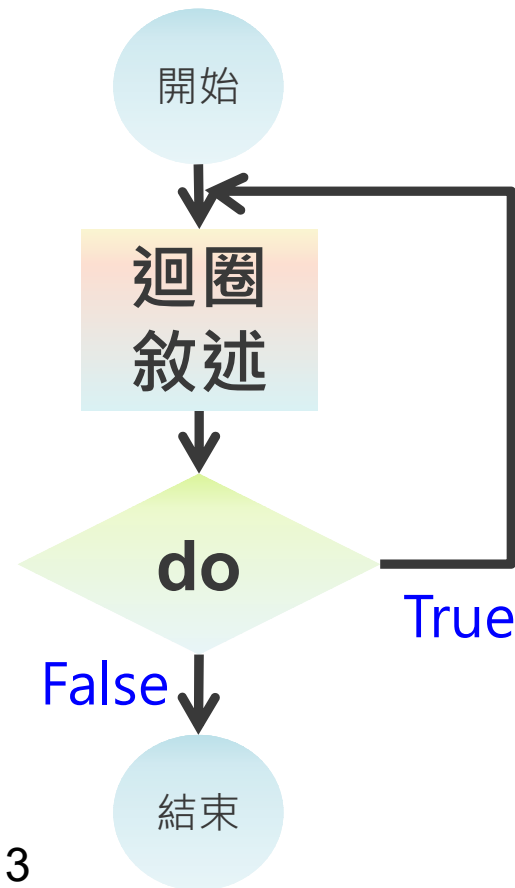
- While後的 () 為條件判斷式，結果為true才執行迴圈，直到條件判斷式結果為false才停止。

```
int i = 0;  
while (i < 10)  
{  
    Console.Write(i + " ");  
    i++;  
}
```



do 迴圈 -1

- **do**為後測式迴圈，執行迴圈程式碼後才判斷條件運算式，若結果為**true**時繼續執行下一次迴圈，為**false**時則停止執行迴圈。
- 至少會執行一次迴圈敘述。
- 必須避免產生無窮迴圈。



do 迴圈-2

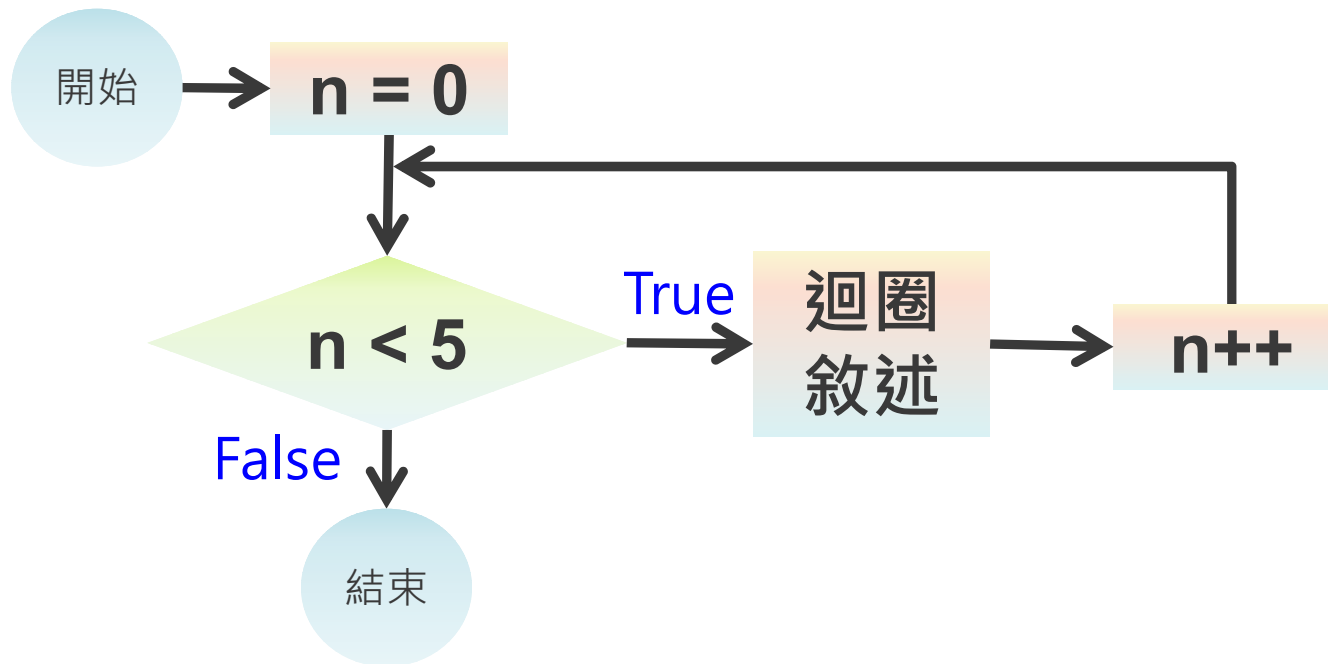
- do 的 { } 中為迴圈敘述，} 後的 while () 為迴圈條件運算式。

```
int i = 0;  
do  
{  
    Console.WriteLine(i++ + " ");  
} while (i < 10);
```



for迴圈-1

- 當已知迴圈執行次數時可以使用。
- 使用for迴圈時必須同時定義啟始值、判斷式、遞增遞減值。
- 可設定一個以上的啟始值(為區域變數)。



for迴圈-2

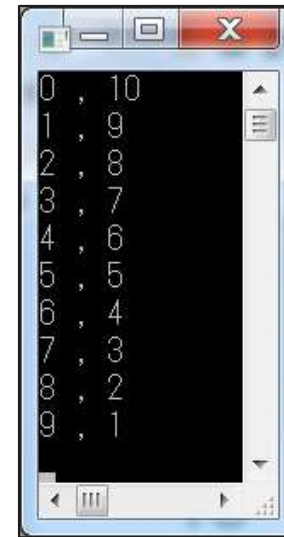
- **for (啟始值 ; 條件式 ; 遞增減值) { //...迴圈敘述... }**

```
for(int i = 0; i < 10; i++)  
{  
    Console.Write(i + " ");  
}
```



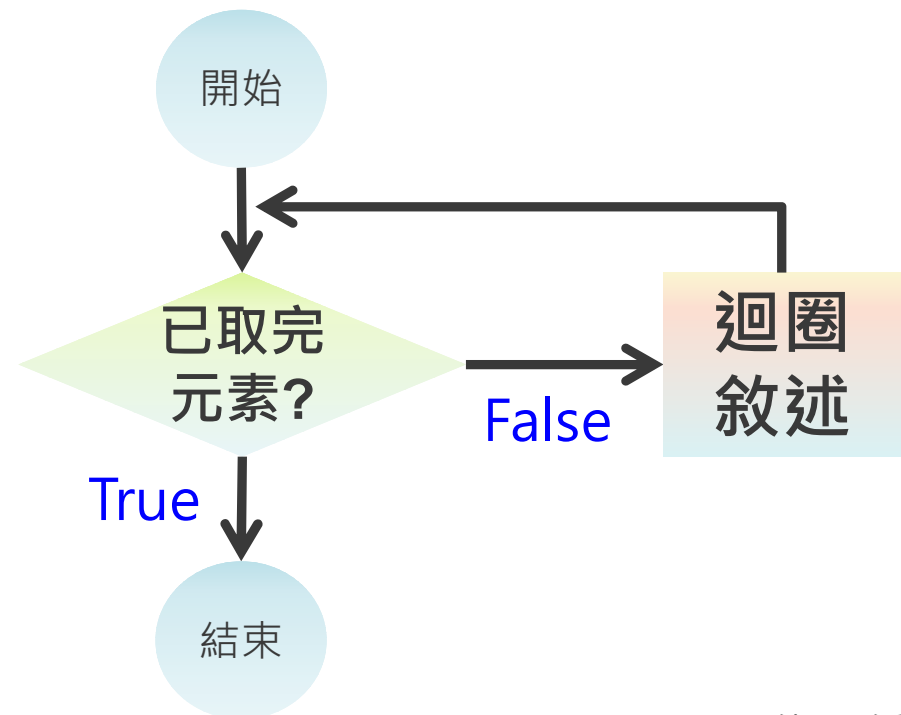
- 啟始值可以宣告一個以上的區域變數。

```
for(int i = 0, r = 10; i < 10; i++, r--)  
{  
    Console.WriteLine(i + ", " + r);  
}
```



foreach 迴圈-1

- 適合用於列舉（**enum**）、陣列（**array**）、集合（**collection**）等具順序性的物件中，能依序存取其中的每個元素。
- 每次執行迴圈時，會自動將集合中的元素逐一取出，並指定給所定義的迴圈變數。
- 迴圈中定義的變數必須與集合元素的型態一致。
- 此變數在迴圈中為唯讀（**read-only**），不能直接修改集合中的元素值。



foreach 迴圈-2

- 當取完所有物件內的元素，迴圈才停止執行。

```
int[] nums = { 0, 1, 2, 3, 4, 5, 7, 8, 9 };  
foreach(int i in nums)  
{  
    Console.Write(i + " ");  
}
```



goto 敘述

- 定義在方法或區塊中，強制跳至指定的位置並執行後續的程式碼。
- **goto**允許跳離迴圈到標籤位置，但不可跳入迴圈。
- 應盡量避免使用。

```
void isEven(int num)
{
    if (num % 2 == 0)
        goto Even;
    else
        goto End;
    Even:
    Console.WriteLine(num + "是偶數喔");
    End:;
}
```

goto敘述使用在switch

```
string isRainy = "下雨";  
string result;  
switch (isRainy)  
{  
    case "下雨":  
        result = "帶傘";  
        break;  
    case "沒下雨":  
        result = "不帶傘";  
        goto default;  
    default:  
        result="介於有和沒有之間";  
        break;  
}
```

break敘述

- **break**敘述用於迴圈，執行到**break**時強制停止執行中的迴圈。

```
int i = 0;
while (i < 10)
{
    Console.Write(i + " ");
    i++;
    if (i > 5)
    {
        break;
    }
}
```



continue 敘述

- **continue**用在迴圈中，當迴圈敘述執行到**continue**時，將跳至下一次迴圈。

```
for (int i = 0; i < 10; i++)  
{  
    if (i % 2 == 0)  
        continue;  
    Console.WriteLine(i);  
}
```

